

# Prompting Claude Opus for High-Performance Results

## Executive summary

The strongest cross-version finding is that **Claude Opus responds best to explicit, well-structured instructions rather than “parameter tuning” tricks**. Anthropic’s current guidance consistently prioritizes: being clear and direct; separating instructions, context, examples, and inputs with XML tags or other explicit sections; assigning a role in the system prompt; using a small set of relevant, diverse examples; and grounding outputs with retrieval, source verification, or direct quotation when accuracy matters. For long inputs, Anthropic specifically recommends placing large documents toward the top of the prompt and the actual question near the end, with structured metadata, because prompt organization materially affects long-context performance. <sup>1</sup>

For **Claude Opus 4.7 and later**, prompt engineering becomes even more central because Anthropic removed reliance on non-default sampling controls such as `temperature`, `top_p`, and `top_k`: setting them to non-default values returns a `400` error, and Anthropic explicitly advises steering behavior through prompting instead. At the same time, Opus 4.7 and 4.8 introduced stronger instruction-following and adaptive-thinking patterns, so prompts written for older models can behave differently, especially if they were vague, contradictory, or dependent on the model “filling in the blanks.” <sup>2</sup>

The most important version-specific changes are these. **Opus 4.5 and 4.6** can be more eager to overengineer or over-delegate, so prompts should explicitly constrain scope and minimalism when you want the simplest acceptable solution. **Opus 4.7** is substantially more literal than earlier Opus releases, which improves predictability but punishes underspecified prompts. **Opus 4.8** keeps the same fundamental prompting style, adds mid-conversation system messages, defaults `effort` to `high`, and improves adaptive thinking, tool triggering, and long-context behavior; existing 4.7 prompts usually transfer, but you should still re-check cost, verbosity, and tool behavior. <sup>3</sup>

Practically, the best prompt strategy for Claude Opus today is to **treat prompting as instruction design plus context engineering**. Anthropic’s engineering team explicitly frames context engineering as the next stage beyond prompt engineering: not just writing good instructions, but curating the right information, tools, examples, and history so the model sees the smallest high-signal context that still lets it succeed. For serious applications, especially agentic or long-context ones, prompt quality should therefore be evaluated together with retrieval quality, tool descriptions, conversation-state management, and output validation. <sup>4</sup>

## Scope and source basis

This report prioritizes **official Anthropic documentation and engineering writing**, then uses technical blog material mainly for corroboration and backward-compatibility commentary, and treats community

discussion only as secondary signal. Anthropic’s current official prompting corpus includes a general “Prompting best practices” reference for the latest models, a model-specific page for **Claude Opus 4.8**, migration guides, tool-use documentation, guardrail guidance, context-window documentation, release notes, and Anthropic engineering posts on context and agent harness design. <sup>5</sup>

For **Opus 4.5 through 4.7**, Anthropic’s official guidance is more distributed than for 4.8. In the sources reviewed, Anthropic currently provides a dedicated model-specific prompting page for **Opus 4.8**, while the main version-specific guidance for **4.5, 4.6, and 4.7** appears in the broader best-practices page, release notes, and migration materials rather than a separate per-model prompting guide. That means some 4.5/4.6/4.7 recommendations below are official, but they are assembled from multiple pages rather than one canonical page. <sup>6</sup>

Anthropic’s app-level system prompts and the Claude API should also be kept conceptually separate. Anthropic publishes system-prompt release notes for **claude.ai** and the mobile apps, and explicitly says those system-prompt updates **do not apply to the Claude API**. So if you are comparing results between Claude’s first-party UI and raw API usage, prompt behavior can differ because the product harness differs. <sup>7</sup>

I did **not** find, in the official sources reviewed, a separate “prompt length limit” or “system-message length limit” beyond the model’s overall context window and output-token constraints. What Anthropic does specify clearly is the available **context window**, **max output**, and in some cases platform-specific differences. Where a more specific limit was not stated in the official material reviewed, I treat it as **unspecified**. <sup>8</sup>

## What Anthropic consistently recommends

Anthropic’s most stable recommendation is that Claude works best when you **say exactly what you want**. Their wording is unusually direct: Claude responds well to clear, explicit instructions, and “above-and-beyond” behavior should be requested explicitly rather than inferred. Anthropic also suggests thinking of Claude as a brilliant new employee who lacks local norms and hidden context. That metaphor is useful because it pushes prompt writers away from implication and toward explicit operating instructions. <sup>9</sup>

The next stable recommendation is **prompt structure**. Anthropic explicitly recommends XML tags for complex prompts because they help Claude distinguish instructions, context, examples, and variable inputs. At the same time, Anthropic’s engineering writing adds an important nuance: strict formatting probably matters less as models improve, but explicit sectioning still helps keep prompts legible and lowers misinterpretation risk. The practical implication is that XML is not magical; it is a high-signal organization device, and it remains especially useful when prompts become long or multi-part. <sup>10</sup>

Anthropic also strongly recommends **few-shot prompting**, but with curation rather than volume. Their current guidance says examples are among the most reliable steering mechanisms for format, tone, and structure; they recommend making them relevant, diverse, and structured, and they explicitly suggest **3–5 examples** as a good starting range. By contrast, Anthropic’s engineering guidance warns against shoving a laundry list of edge cases into the prompt. The right pattern is a compact set of canonical examples that portray desired behavior without flooding the context window. <sup>11</sup>

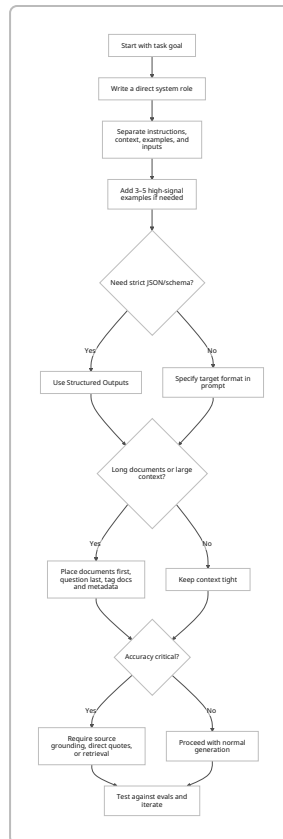
A **role in the system prompt** remains a core Anthropic recommendation. Even a one-sentence role can sharpen tone and behavior. This is especially useful when you want Claude to maintain a professional stance, domain-specific vocabulary, conservative uncertainty handling, or a consistent task posture such as “senior research analyst,” “static code reviewer,” or “enterprise support agent.” <sup>12</sup>

For **long context**, Anthropic is unusually explicit: for large inputs, put the documents near the top of the prompt and the actual question near the end; for multi-document work, wrap documents and metadata in structured tags. Anthropic states that queries at the end can improve response quality by up to 30% in their tests, especially with complex multi-document inputs. This is not just prompt-writing advice; it is context-ordering advice. Anthropic’s broader engineering work reinforces the same point by arguing that context is a finite, degradable resource and that more context is not automatically better. <sup>13</sup>

For **accuracy-sensitive work**, Anthropic recommends grounding rather than “trusting reasoning alone.” Their guardrail docs say to permit “I don’t know,” and for long-document work they recommend extracting direct quotes before higher-level reasoning. Their consistency docs recommend retrieval or fixed contextual grounding when responses should remain anchored to a stable knowledge set. In other words, if truth matters, the best Claude prompt is rarely “think harder”; it is usually “use these sources, quote them first, then analyze.” <sup>14</sup>

For **format control**, Anthropic advises two complementary strategies. First, specify the target format precisely with JSON, XML, or a custom template. Second, if you need guaranteed schema conformance, do not rely on prompting alone: use **Structured Outputs**. Anthropic is explicit that Structured Outputs are the correct tool when you need valid JSON that conforms to a schema, while prompt engineering remains the right tool for looser stylistic or analytical constraints. Anthropic also notes a useful micro-pattern: tell Claude what to do instead of what not to do. <sup>15</sup>

A final stable recommendation is **test prompts with evaluations, not intuition alone**. Anthropic’s docs encourage defining measurable success criteria and building evals across the dimensions you actually care about: fidelity, consistency, relevance, tone, privacy, context use, latency, and price. That is important because Anthropic’s own engineering writeups show that seemingly reasonable prompt changes can improve one behavior but degrade real task performance elsewhere. <sup>16</sup>



This flow is a synthesis of Anthropic’s prompt-structure, long-context, formatting, and guardrail guidance.

17

## Cross-version behavior notes

The most consequential cross-version break is that **Opus 4.7 and later reject non-default sampling parameters**. Anthropic’s migration guide and Messages docs say that setting `temperature`, `top_p`, or `top_k` to non-default values returns a `400` error on Opus 4.7 and Opus 4.8, and they explicitly recommend omitting those fields and using prompting to steer behavior instead. This matters because some older Claude prompting advice on the broader internet still assumes temperature-tuning is available. For Opus 4.7+, the official answer is: **prompt, don’t tune**. 18

Another major change is the shift from manual extended-thinking budgets toward **adaptive thinking plus effort**. Anthropic’s current docs say Opus 4.7 and 4.8 no longer support the older `thinking: {type: "enabled", budget_tokens: N}` mode; instead, use `thinking: {type: "adaptive"}` and control depth with the `effort` parameter. Opus 4.8 keeps thinking **off by default unless you explicitly enable adaptive thinking**, but once enabled it decides turn-by-turn whether reasoning is needed. Anthropic recommends at least `high` effort for intelligence-sensitive workloads, and specifically recommends `xhigh` for coding and agentic use cases. 19

Anthropic’s docs also flag a subtle but important **Opus 4.5 behavior**: when extended thinking is disabled, Opus 4.5 is “particularly sensitive” to the word **“think”** and related variants. Anthropic’s recommendation is

to prefer alternatives such as “consider,” “evaluate,” or “reason through” in that case. This is one of the most concrete version-specific prompting quirks Anthropic has documented, and it is easy to miss if you only read generic Claude advice. <sup>20</sup>

**Opus 4.5 and 4.6** are also explicitly documented as having a tendency toward **overengineering**. Anthropic says these models may create extra files, add abstractions, or build flexibility you did not ask for. Their recommended remedy is to impose minimalism explicitly: scope boundaries, anti-refactor rules, no hypothetical future-proofing, and “only make changes directly requested or clearly necessary.” This is official guidance, not merely community folklore. <sup>21</sup>

**Opus 4.6** introduced a stronger adaptive-thinking and subagent posture. Anthropic’s best-practices page says Opus 4.6 can have a “strong predilection” for subagents, sometimes in cases where a simple direct tool call would have been enough. Anthropic’s advice is to define when delegation is and is not warranted. In practice, that means prompts should contain a small operational policy for decomposition rather than assuming the model will naturally pick the cheapest path. <sup>22</sup>

**Opus 4.7** is the version where Anthropic most clearly warns prompt writers to recalibrate. Its launch materials say instruction-following improved enough that prompts written for earlier models may now yield unexpected results because Opus 4.7 takes instructions more literally where earlier models were looser or skipped parts. This exact behavior is echoed in Hacker News discussions around the launch, which broadly align with Anthropic’s own warning. The practical takeaway is that prompts for 4.7 should be more precise, less contradictory, and less dependent on tacit defaults. <sup>23</sup>

**Opus 4.8** is mostly continuity plus ergonomics. Anthropic says there are no breaking API changes relative to 4.7, but there are behavior changes worth checking: default `effort: high`, mid-conversation system messages, lower minimum cacheable prompt length, better tool triggering, fewer wasted thinking tokens when adaptive thinking is enabled, and better compaction / long-context handling. The prompt style that works on 4.7 generally transfers to 4.8, but long-running systems benefit from taking advantage of 4.8-specific features such as mid-conversation system messages and improved adaptive reasoning. <sup>24</sup>

Assistant-message **prefill** is a major migration boundary. Anthropic’s docs say prefilling is not supported on Opus 4.6, 4.7, or 4.8, and requests using it return a `400` error. For 4.5-era workflows, Anthropic’s release notes also show that Structured Outputs became GA for Opus 4.5 in January 2026, and Anthropic now recommends Structured Outputs, system-prompt instructions, or `output_config.format` instead of prefilling on current models. <sup>25</sup>

The table below summarizes the most relevant version-sensitive prompting differences.

| Version band | High-impact prompting behavior                                    | What to do                                                            | What to avoid                                                | Source        |
|--------------|-------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------|---------------|
| Opus 4.5     | Can be sensitive to “think” wording when extended thinking is off | Prefer “consider,” “evaluate,” or “reason through” if thinking is off | Blindly reusing chain-of-thought phrasing from older prompts | <sup>20</sup> |

| Version band     | High-impact prompting behavior                                                              | What to do                                                                                   | What to avoid                                                     | Source |
|------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------|
| Opus 4.5 and 4.6 | Can overengineer and add unnecessary abstractions                                           | Add explicit minimalism and scope instructions                                               | Leaving “improve this” or “make it robust” underspecified         | 21     |
| Opus 4.6         | Stronger adaptive thinking and a tendency to overuse subagents in some agentic flows        | State when delegation is warranted and when direct work is preferred                         | Assuming the model will always choose the simplest execution path | 26     |
| Opus 4.7         | Much more literal instruction following                                                     | Remove ambiguity, contradictions, and hidden assumptions                                     | Relying on the model to infer omitted constraints                 | 27     |
| Opus 4.7 and 4.8 | Non-default <code>temperature</code> , <code>top_p</code> , <code>top_k</code> are rejected | Steer with prompting and <code>effort</code>                                                 | Sampling-parameter tuning workflows                               | 28     |
| Opus 4.7 and 4.8 | Manual thinking budgets removed; adaptive thinking is the supported path                    | Use <code>thinking: {type: "adaptive"}</code> plus <code>output_config.effort</code>         | <code>budget_tokens</code> -based thinking configs                | 19     |
| Opus 4.8         | Adds mid-conversation system messages and better long-context ergonomics                    | Update instructions via system messages instead of rebuilding full history where appropriate | Re-sending a whole long prompt when only new policy is needed     | 29     |
| Opus 4.6+        | Prefill unsupported                                                                         | Use Structured Outputs or explicit response-format instructions                              | Assistant prefill techniques from older prompting playbooks       | 25     |

## Prompt patterns and templates

The highest-yield prompt pattern for Claude Opus is a **four-part structure**: role, task contract, grounded context, and output contract. Anthropic’s docs strongly support each piece independently, and taken together they produce prompts that are easy for Claude to parse and easy for you to evaluate. The role sets posture; the task contract defines success and failure; the grounded context supplies only the information needed; and the output contract controls structure, detail level, and uncertainty policy. 30

A good default template for analytical or research work looks like this:

```

<system>
You are a rigorous research analyst. Be explicit about uncertainty.
Use only the provided sources unless I explicitly ask for outside knowledge.
If the evidence is insufficient, say so plainly.
</system>

<task>
Answer the user's question. Identify the main conclusion, the strongest
supporting evidence,
important counterevidence, and any unresolved uncertainty.
</task>

<success_criteria>
- Accurate and source-grounded
- Clear decision-oriented conclusion
- Distinguish fact from inference
- No unsupported claims
</success_criteria>

<context>
  <documents>
    <document index="1">
      <source>...</source>
      <document_content>...</document_content>
    </document>
    <document index="2">
      <source>...</source>
      <document_content>...</document_content>
    </document>
  </documents>
</context>

<instructions>
Before finalizing, verify the answer against the supplied evidence.
If the documents disagree, explain the disagreement rather than hiding it.
</instructions>

<output_format>
Return:
1. A short answer
2. A deeper analysis
3. Open questions
</output_format>

<user_question>
{{question}}
</user_question>

```

This template follows Anthropic’s guidance on roles, XML structure, explicit success criteria, source verification, and self-checking. For API use, if you require machine-validated JSON rather than prose, replace the prose output contract with Structured Outputs rather than trying to prompt your way into guaranteed schema compliance. <sup>31</sup>

For **coding and implementation tasks**, Claude Opus benefits from a stronger execution contract. Anthropic’s docs suggest telling the model not to speculate about code it has not read, not to optimize for passing tests alone, and not to add abstractions or helper scripts unless truly necessary. A strong coding template therefore looks like this:

```
<system>
You are a senior software engineer working in an existing codebase.
Read before answering. Never speculate about files you have not opened.
Prefer the minimum correct change.
</system>

<task>
Implement the requested change.
</task>

<constraints>
- Do not refactor unrelated code
- Do not add abstractions for one-off needs
- Do not hard-code values just to satisfy tests
- If the task is infeasible or the tests are wrong, explain that directly
</constraints>

<quality_bar>
Before finishing, verify:
- the change solves the stated problem generally
- the implementation matches local codebase patterns
- no unnecessary files or helper scripts remain
</quality_bar>

<request>
{{request}}
</request>
```

This structure maps directly onto Anthropic’s guidance on minimalism, anti-hard-coding, and investigate-before-answering behavior. It is especially important for Opus 4.5 and 4.6, which Anthropic says tend toward overengineering, and still useful for 4.7/4.8 because their stronger autonomy can otherwise create unnecessary surface area. <sup>32</sup>

## Good and bad prompt examples

The contrast below is the easiest way to see what “Anthropic-style prompting” actually means in practice.

## Bad

Review this architecture doc and tell me what you think.

## Good

```
<system>
You are a principal architecture reviewer. Be concrete and critical.
</system>

<context>
  <document>
    <source>architecture_doc</source>
    <document_content>{{doc}}</document_content>
  </document>
</context>

<task>
Review the proposal for correctness, scalability, operational risk, and
migration risk.
</task>

<output_format>
Return four sections:
- Key strengths
- Critical risks
- Gaps or ambiguities
- Recommended decision
</output_format>

<instructions>
Quote the exact passages that support each critical risk.
If evidence is missing, say "insufficient information" rather than guessing.
</instructions>
```

The good version is better because it gives Claude a role, a review lens, a grounded input structure, an output format, and an uncertainty policy. Anthropic's docs consistently recommend each of those moves.

33

## Bad

Summarize these docs and keep it under 200 words.

## Good

```

<documents>
  <document index="1">
    <source>spec_a</source>
    <document_content>{{doc_a}}</document_content>
  </document>
  <document index="2">
    <source>spec_b</source>
    <document_content>{{doc_b}}</document_content>
  </document>
</documents>

<task>
Compare the documents and summarize the most decision-relevant differences.
</task>

<instructions>
Write exactly two short paragraphs.
The first paragraph should state the high-level differences.
The second paragraph should state implications for implementation.
</instructions>

<question>
What are the most important differences?
</question>

```

This is better because Anthropic explicitly says paragraph or sentence constraints are usually more effective than exact word-count limits, and because long-context organization matters. <sup>34</sup>

## Bad

Think step by step and solve this.

## Good for Opus 4.5 when thinking is off

Reason through the problem carefully before answering.  
Then provide the final answer separately.

## Better for Opus 4.7 and 4.8 when you want official reasoning support

```

{
  "thinking": {"type": "adaptive"},

```

```
"output_config": {"effort": "high"}
}
```

The nuance is that Anthropic explicitly warns that Opus 4.5 can be sensitive to “think” phrasing when thinking is off, while Opus 4.7 and 4.8 are designed around adaptive thinking plus effort. Anthropic also notes that overly prescriptive human-written step plans are often worse than higher-level guidance such as “think thoroughly,” because Claude’s reasoning can exceed the human-authored procedure. <sup>35</sup>

## Technique comparison table

| Technique                             | When to use                                    | Expected effect                            | Example prompt fragment                                          | Notes                                                                                               |
|---------------------------------------|------------------------------------------------|--------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Direct task contract                  | Always                                         | Better alignment and less hidden inference | State the main conclusion, supporting evidence, and uncertainty. | Anthropic’s baseline recommendation. <sup>9</sup>                                                   |
| System role                           | Almost always                                  | Better tone, posture, and domain framing   | You are a senior incident-response analyst.                      | Even one sentence helps. <sup>36</sup>                                                              |
| XML or explicit sections              | Multi-part prompts, long context, examples     | Less ambiguity, easier parsing             | <instructions>...</instructions>                                 | Still recommended even though exact formatting may matter a bit less on newer models. <sup>10</sup> |
| Few-shot examples                     | When output style or reasoning pattern matters | Higher consistency and accuracy            | <examples><example>...</example></examples>                      | Start with 3–5 strong examples. <sup>37</sup>                                                       |
| Long-context ordering                 | Large documents, multi-doc synthesis           | Better retrieval and answer quality        | Put docs first, question last                                    | Anthropic says question-at-end can materially help. <sup>38</sup>                                   |
| Grounding through quotes or retrieval | High-stakes fact work                          | Fewer hallucinations                       | Extract direct quotes first, then analyze.                       | Strongly recommended for long documents and fixed knowledge sets. <sup>14</sup>                     |

| Technique                     | When to use                                       | Expected effect                                    | Example prompt fragment                                         | Notes                                                                             |
|-------------------------------|---------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Adaptive thinking plus effort | Hard reasoning, coding, agentic workflows on 4.7+ | Better reasoning depth with controllable token use | <code>thinking: adaptive</code> + <code>effort: high</code>     | Use prompt guidance, not sampling params, on 4.7+. <sup>39</sup>                  |
| Self-check instruction        | Math, coding, structured analysis                 | Catches avoidable mistakes                         | <code>Before finishing, verify against these criteria...</code> | Anthropic explicitly recommends self-checking. <sup>20</sup>                      |
| Structured Outputs            | Strict JSON / schema workflows                    | Guaranteed schema conformance                      | Define schema in <code>output_config.format</code>              | Prefer over “JSON-only” prompt hacks. <sup>40</sup>                               |
| Prompt chaining               | When you need inspectable intermediate steps      | Better control and debuggability                   | draft → critique → revise                                       | Still useful even with better internal reasoning. <sup>22</sup>                   |
| Minimalism constraints        | Coding tasks, especially 4.5 and 4.6              | Less overengineering and scope creep               | <code>Only make changes directly requested.</code>              | Officially recommended for Opus 4.5/4.6. <sup>21</sup>                            |
| Explicit tool descriptions    | Tool-using agents                                 | Better tool selection and lower ambiguity          | Detailed tool descriptions and well-named params                | Anthropic says small tool-description refinements can matter a lot. <sup>41</sup> |

Illustrative official visuals that pair well with the table are Anthropic’s **context-window diagram** and its **prompt-engineering-vs-context-engineering** figure. Both are helpful because they make clear that prompt structure and context curation are part of the same control surface. <sup>42</sup>

## Troubleshooting common failure modes

The first failure mode is **Claude missed part of the assignment**. On current Opus models, especially 4.7 and 4.8, the usual cause is not “weak intelligence” so much as **underspecified success criteria** or instructions that implicitly rely on your internal defaults. Anthropic’s own language says Claude is like a brilliant new employee without your local context. The fix is to define the output contract explicitly, state the dimensions that matter, and, where useful, show one or two examples of a successful result. If you are

migrating from earlier prompts to 4.7+, also remove contradictory hints and vague “do your best” language, because a more literal model is less likely to repair ambiguity for you. 43

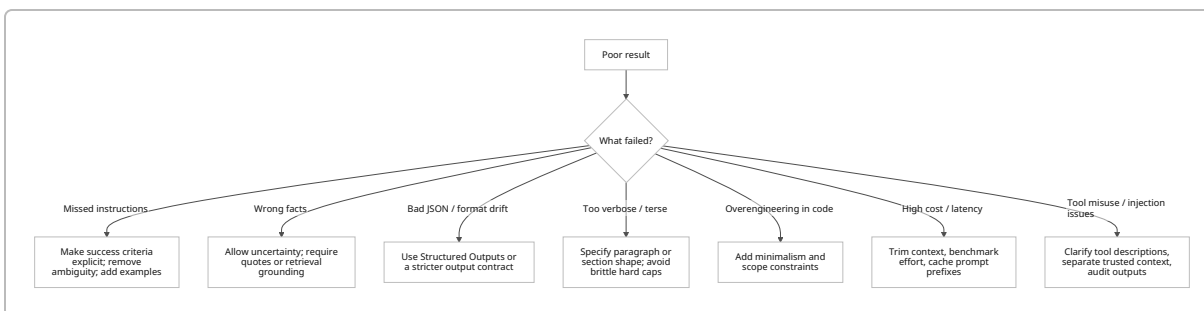
The second failure mode is **Claude is too verbose, or too terse**. Anthropic’s guidance is nuanced here. Their docs recommend clear output-format instructions, paragraph-count or sentence-count constraints rather than word-count micromanagement, and explicit post-tool summary instructions if you want more visibility. But Anthropic’s own postmortem is a warning against overconstraining length: a system-prompt instruction to keep tool-call text under 25 words and final responses under 100 words hurt coding quality badly enough that Anthropic reverted it. The lesson is to shape verbosity at the level of communicative intent, not with hard caps unless brevity is genuinely the task. 44

The third failure mode is **hallucination or unjustified confidence**. Anthropic’s official fixes are straightforward and strong: permit “I don’t know,” ground in retrieval or a fixed reference set, and for long documents require extraction of direct quotes before analysis. In coding contexts, Anthropic even suggests the instruction “Never speculate about code you have not opened.” This is a powerful general pattern: when you want truth, force the model to separate evidence collection from synthesis. 45

The fourth failure mode is **bad JSON or unstable output formatting**. Anthropic’s answer here is to stop overinvesting in clever prompt wording and use the platform feature built for the job: Structured Outputs. Their docs explicitly say to use Structured Outputs when you need valid JSON matching a schema. If you are on Opus 4.6+, assistant prefills are not the right workaround, because they are unsupported. Also note one official incompatibility: Anthropic says citations and Structured Outputs cannot currently be used together in the same response. 46

The fifth failure mode is **too much cost or latency**. Anthropic recommends solving performance after you first solve correctness. Once the prompt works, trim context, ask for shorter responses by paragraph or sentence count, use `max_tokens` only as a hard stop when appropriate, enable prompt caching for repeated fixed prefixes, and consider streaming to improve perceived responsiveness. For Opus 4.7 and 4.8, output-token use is also affected by adaptive thinking and effort, so you should explicitly benchmark the effort level that gives the best quality-per-token for your workload. 47

The sixth failure mode is **prompt leaks or prompt injection** in tool-using or agentic systems. Anthropic distinguishes between jailbreaks and prompt injections and recommends additional guardrails: separate context from queries, monitor outputs, avoid unnecessary proprietary details, and prefer monitoring or post-processing before adding very elaborate leak-resistant prompt logic. Anthropic explicitly warns that overcomplicated leak-proofing can degrade model performance. In practice, that means the best defense is usually a combination of clearer separation, tool and retrieval boundaries, and output auditing—not an ever-larger defensive prompt. 48



This decision tree synthesizes Anthropic’s official guidance on consistency, hallucinations, latency, tool use, and prompt-leak mitigation. <sup>49</sup>

## Checklist and prioritized do’s and don’ts

A short drafting checklist that is faithful to Anthropic’s current guidance looks like this:

- Define the role in one sentence.
- State the task and success criteria explicitly.
- Separate instructions, context, examples, and user input into distinct sections.
- If style or structure matters, include 3–5 high-signal examples.
- If working over long documents, place documents first and the question last.
- If accuracy matters, require quotes, retrieval, or explicit source verification.
- If strict JSON is required, use Structured Outputs instead of prompt-only control.
- For Opus 4.7 and later, steer behavior with prompting and `effort`, not non-default sampling parameters.
- For coding tasks, explicitly say whether to optimize for minimal change, general correctness, or both.
- Test the prompt against eval cases before shipping. <sup>50</sup>

The most important **do’s** are straightforward. **Do** be explicit about success. **Do** give Claude a role. **Do** organize prompts rather than writing one large blob. **Do** use examples sparingly but intentionally. **Do** constrain scope when you want minimal changes. **Do** separate retrieval or quoting from synthesis when correctness matters. **Do** benchmark prompts with real evals, because Anthropic’s own experience shows prompt changes can have non-obvious regressions. <sup>51</sup>

The highest-priority **don’ts** are equally clear. **Do not** rely on Claude to infer unspoken success criteria. **Do not** assume old sampling-parameter workflows still apply to Opus 4.7+. **Do not** use assistant prefills on Opus 4.6 and later. **Do not** equate “more context” with “better context.” **Do not** dump every possible edge case into the prompt instead of curating examples. **Do not** use brittle word-count micromanagement as a substitute for a real output contract. **Do not** overcomplicate prompt-leak defenses without testing the performance tradeoff. <sup>52</sup>

Community discussion largely reinforces these official conclusions, though it should be treated as anecdotal rather than definitive. Hacker News threads around Opus 4.7 repeatedly echoed Anthropic’s official note that the model follows instructions more literally than earlier versions, and Reddit discussions around Claude prompting repeatedly point back to Anthropic’s XML-tag guidance as one of the more reliable

practical techniques. Those signals are useful mainly because they match the official docs, not because they override them. 53

## Open questions and limitations

Anthropic's official docs are currently strongest for **Opus 4.8** and for general latest-model guidance. For **Opus 4.5**, there is official material, but it is more fragmented, and some present-day docs emphasize migration away from 4.5-era techniques rather than maintaining a separate 4.5-specific playbook. I therefore have higher confidence in the 4.7/4.8-specific recommendations than in any attempt to finely distinguish every 4.5 nuance beyond what Anthropic explicitly documented. 54

I also did **not** find, in the official sources reviewed, a separate hard cap for prompt length or system-message length beyond the documented context windows and output limits. Where the docs were silent, I have labeled the detail **unspecified** rather than inferring a hidden limit. Similarly, some older third-party prompting guides still recommend tactics such as assistant prefills or temperature tuning; those are now partially obsolete for current Opus models and should be treated carefully against Anthropic's current migration and API docs. 55

The best bottom-line recommendation is therefore simple: if you want optimal results from Claude Opus, especially **4.7 and 4.8**, design prompts as **explicit operating contracts over carefully curated context**, validate them with evals, and use official platform features such as **adaptive thinking, effort, Structured Outputs, prompt caching, and mid-conversation system messages** where they fit. That strategy is the one most consistently supported by Anthropic's official documentation, engineering writing, and the secondary evidence surveyed here. 56

---

1 3 6 9 10 11 12 13 17 20 21 22 26 30 31 32 33 35 36 37 38 43 44 50 51 52 56 Prompting best practices - Claude API Docs

<https://docs.anthropic.com/en/prompt-library/library>

2 18 19 24 28 39 Migration guide - Claude API Docs

<https://docs.anthropic.com/en/docs/about-claude/models/migrating-to-claude-4>

4 Effective context engineering for AI agents \ Anthropic

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

5 Prompting best practices - Claude API Docs

[https://docs.anthropic.com/en/prompt-library/library?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/prompt-library/library?utm_source=chatgpt.com)

7 System Prompts - Claude API Docs

<https://docs.anthropic.com/en/release-notes/system-prompts>

8 42 55 Context windows - Claude API Docs

<https://docs.anthropic.com/en/docs/build-with-claude/context-windows>

14 45 Reduce hallucinations - Claude API Docs

<https://docs.anthropic.com/en/docs/test-and-evaluate/strengthen-guardrails/reduce-hallucinations>

15 40 46 49 Increase output consistency - Claude API Docs

<https://docs.anthropic.com/en/docs/test-and-evaluate/strengthen-guardrails/increase-consistency>

16 Define success criteria and build evaluations - Claude API Docs

<https://docs.anthropic.com/en/docs/build-with-claude/develop-tests>

23 27 Introducing Claude Opus 4.7

[https://www.anthropic.com/news/claude-opus-4-7?utm\\_source=chatgpt.com](https://www.anthropic.com/news/claude-opus-4-7?utm_source=chatgpt.com)

25 Using the Messages API - Claude API Docs

<https://docs.anthropic.com/en/api/messages-examples>

29 54 What's new in Claude Opus 4.8 - Claude API Docs

<https://docs.anthropic.com/en/docs/about-claude/models/whats-new-claude-4-8>

34 47 Reducing latency - Claude API Docs

<https://docs.anthropic.com/en/docs/test-and-evaluate/strengthen-guardrails/reduce-latency>

41 Define tools - Claude API Docs

<https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

48 Mitigate jailbreaks and prompt injections - Claude API Docs

<https://docs.anthropic.com/en/docs/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks>

53 Claude Opus 4.7

[https://news.ycombinator.com/item?id=47793411&utm\\_source=chatgpt.com](https://news.ycombinator.com/item?id=47793411&utm_source=chatgpt.com)